FIAP

David de Sá Tomaz RM: 570348
Silvio Arthur Soares de Oliveira RM: 570382
Antuny Marques de Menezes RM: 572107
Eric Yuiti Ito Nissi RM: 573495
Vinícius Pelogia do Nascimento RM: 572675

IGNITION ZERO: O INÍCIO DA AURORA

RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

São Paulo 2026

1. ORGANIZAÇÃO E DESCRIÇÃO DA TELEMETRIA **3**
1.1 Definição das Variáveis de Entrada 3
1.2 Critérios de Segurança e Faixas Operacionais 4
2. ALGORITMO DE VERIFICAÇÃO (PSEUDOCÓDIGO) **5**
2.1 Estrutura Lógica do Sistema 5
2.2 Pseudocódigo do Programa 5
2.3 Fluxograma 6
2.4 Identificação de Anomalias (Análise Assistida) 6
3. IMPLEMENTAÇÃO DO SCRIPT EM PYTHON **7**
3.1 Estrutura do Código-Fonte 7
3.2 Descrição das Funcionalidades 10
4. ANÁLISE ENERGÉTICA E AUTONOMIA **12**
4.1 Fundamentação Matemática 12
4.2 Metodologia de Cálculo e Eficiência 12
4.3 Cenário de Simulação 12
5. ANÁLISE ASSISTIDA POR INTELIGÊNCIA ARTIFICIAL (IA) **13**
5.1 Classificação de Dados e Identificação de Anomalias 13
5.2 Sugestões de Risco e Tomada de Decisão 13
6. REFLEXÃO CRÍTICA: ÉTICA, SOCIEDADE E SUSTENTABILIDADE **14**
6.1 Ética e Responsabilidade Social (ISO 26000) 14
6.2 Sustentabilidade e o Triple Bottom Line (TBL) 14
6.3 Sustentabilidade Tecnológica e Green IT 15
7. REFERÊNCIAS **15**

1. ORGANIZAÇÃO E DESCRIÇÃO DA TELEMETRIA

A telemetria, definida como o conjunto de dados enviados automaticamente pelos sistemas para monitoramento técnico em tempo real, constitui o pilar fundamental para a segurança operacional da Missão Aurora Siger. Nesta etapa, estabeleceu-se a estruturação das variáveis críticas que compõem o sistema embarcado da nave, assegurando que a leitura dos sensores seja interpretada de forma precisa pelo algoritmo de decisão.

1.1 Definição das Variáveis de Entrada

Para a análise de viabilidade da decolagem, foram selecionados seis parâmetros fundamentais. A tabela 1 detalha a tipagem de dados e as unidades de medida aplicadas, garantindo a padronização necessária para a implementação em linguagem Python.

Tabela 1 – Especificação Técnica dos Dados de Telemetria

Parâmetro	**Tipo de Dado**	**Unidade**	**Descrição Técnica**
Temperatura Interna	Float	°C	Monitoramento do ambiente de hardware e suporte à vida.
Temperatura Externa	Float	°C	Verificação das condições atmosféricas e térmicas externas.
Integridade Estrutural	Booleano	0 / 1	Indicador binário de falha (0) ou integridade (1) do casco.
Nível de Energia	Inteiro	%	Percentual de carga acumulada no sistema de baterias.

Pressão dos Tanques	Float	bar	Pressão interna dos reservatórios de propelente.
Status dos Módulos	Booleano	0 / 1	Verificação de disponibilidade dos sistemas críticos.

1.2 Critérios de Segurança e Faixas Operacionais

A lógica de decolagem fundamenta-se em janelas de segurança predefinidas. O desvio de qualquer um desses parâmetros em relação aos valores nominais resulta na condição de "Anomalia Operacional", gatilho imediato para o aborto da missão.

As faixas de segurança estabelecidas para este projeto são:

- **Térmica:** A temperatura interna deve manter-se entre **15°C e 35°C**, enquanto a externa deve situar-se entre **-50°C e 60°C**.
- **Energética:** O sistema exige um mínimo de **80%** de carga para garantir autonomia em manobras de contingência.
- **Pressão:** O intervalo nominal de operação dos tanques está fixado entre **120 bar e 150 bar**.
- **Sistemas:** Tanto a integridade estrutural quanto o status dos módulos críticos devem retornar obrigatoriamente o valor lógico **1 (Ativo)**.

Este conjunto de regras booleanas permite que o processador central execute uma verificação cruzada (utilizando operadores lógicos *AND*), onde a decolagem só é autorizada se a totalidade dos critérios for satisfeita simultaneamente.

2. ALGORITMO DE VERIFICAÇÃO (PSEUDOCÓDIGO)

O algoritmo de verificação pré-decolagem da Missão Aurora Siger foi desenvolvido para atuar como um sistema de suporte à decisão. Ele processa dados multivariáveis de telemetria e aplica filtros de segurança baseados em limites técnicos operacionais.

2.1 Estrutura Lógica do Sistema

O fluxo lógico utiliza uma estrutura de **desvio condicional composto**. O sistema primeiro realiza o cálculo da eficiência energética e, em seguida, valida cada sensor individualmente. A autorização de voo é concedida apenas através de uma operação lógica E (AND) global, onde a falha em um único parâmetro resulta no cancelamento da decolagem.

2.2 Pseudocódigo do Programa

```
ALGORITMO Verificar_Missao_Aurora
VAR
    // Variáveis de Telemetria
    perc_atual, temp_int, temp_ext, pressao: REAL
    int_estrutural: INTEIRO
    mod_nav, mod_com, mod_prop: CARACTERE

    // Variáveis de Processamento
    energia_util, autonomia_restante: REAL
    logica_seguranca, sistemas_ok: LOGICO

INÍCIO
    // 1. Coleta de Dados (Entradas)
    ESCREVA "Digite a carga atual (%): "
    LEIA perc_atual
    ESCREVA "Digite a temperatura interna e externa: "
    LEIA temp_int, temp_ext
    ESCREVA "Digite a pressão do tanque: "
    LEIA pressao
    ESCREVA "Status estrutural (1-OK / 0-FALHA): "
    LEIA int_estrutural
    ESCREVA "Status dos módulos (Navegação, Comunicação, Propulsão): "
    LEIA mod_nav, mod_com, mod_prop

    // 2. Cálculos Energéticos (Baseados em E = P * t)
    energia_util <- (500 * (perc_atual / 100)) * 0.92 // Desconto de 8% de perda
    autonomia_restante <- energia_util - 300 // Consumo de decolagem

    // 3. Verificação de Sistemas Críticos
    SE (mod_nav == "OK" E mod_com == "OK" E mod_prop == "OK") ENTÃO
        sistemas_ok <- VERDADEIRO
    SENÃO
        sistemas_ok <- FALSO
    FIM_SE

    // 4. Validação da Lógica de Segurança (Checklist)
    SE (temp_int >= 15 E temp_int <= 35) E
       (temp_ext >= -50 E temp_ext <= 60) E
       (pressao >= 120 E pressao <= 150) E
```

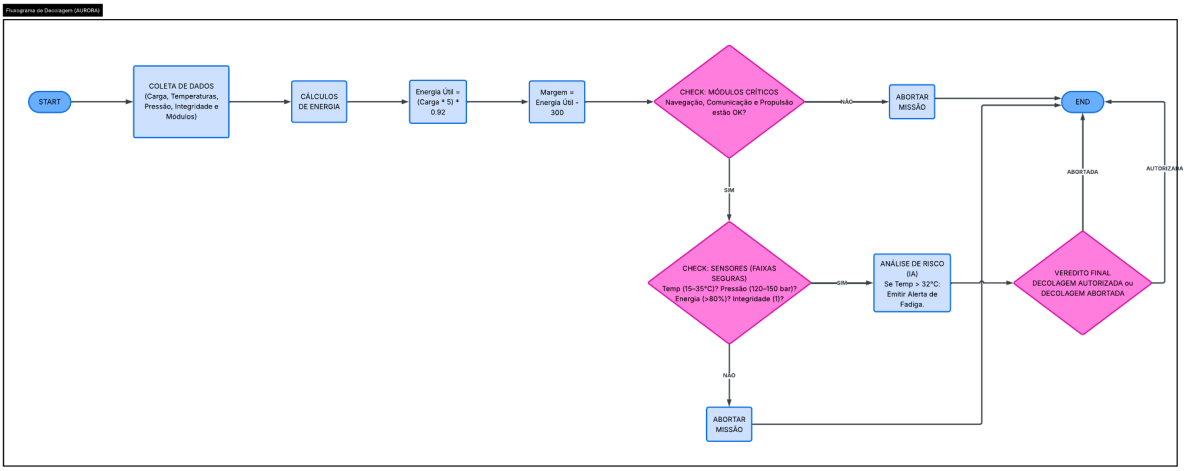
```
        (perc_atual >= 80) E
        (energia_util >= 300) E
        (int_estrutural == 1) E
        (sistemas_ok == VERDADEIRO) ENTÃO

         logica_seguranca <- VERDADEIRO
    SENÃO
         logica_seguranca <- FALSO
    FIM_SE

    // 5. Veredito Final
   ESCREVA "--- RELATÓRIO DE SEGURANÇA ---"
    ESCREVA "Energia Útil Disponível: ", energia_util, " kWh"

    SE (logica_seguranca == VERDADEIRO) ENTÃO
         ESCREVA "STATUS: !!! PRONTO PARA DECOLAR !!!"
    SENÃO
         ESCREVA "STATUS: !!! DECOLAGEM ABORTADA !!!"
         ESCREVA "MOTIVO: Falha nos critérios de segurança."
    FIM_SE
FIM
```

2.3 Fluxograma



2.4 Identificação de Anomalias (Análise Assistida)

O algoritmo incorpora uma camada de monitoração preventiva. Caso a variável temp_int situe-se no intervalo entre **32°C e 35°C**, o sistema emite um alerta de "Risco de Fadiga Térmica", permitindo que a equipa de solo tome decisões informadas antes que o limite de segurança seja violado e o cancelamento automático seja disparado.

3. IMPLEMENTAÇÃO DO SCRIPT EM PYTHON

Nesta etapa, a lógica de programação foi traduzida para a linguagem Python, utilizando uma abordagem funcional para simular o sistema de processamento de dados da nave. O script foi projetado para ser interativo, permitindo que o operador insira os dados de telemetria e receba um diagnóstico imediato sobre a viabilidade do voo.

3.1 Estrutura do Código-Fonte

O código utiliza variáveis de ponto flutuante (*float*) para garantir a precisão nas medições térmicas e de pressão, e estruturas condicionais encadeadas para a tomada de decisão. Abaixo, apresenta-se a implementação completa:

```
# ================================================================

# PROJETO MISSION AURORA SIGER - SISTEMA DE VERIFICAÇÃO DE VOO

# ================================================================



# === 1. DADOS BASE (PARAMETRIZAÇÃO) ===

capacidade_total_kwh = 500  # Capacidade total da bateria

taxa_perda_percent = 8      # Perda por eficiência térmica/transmissão

consumo_decolagem_kwh = 300 # Energia necessária para a ignição



# Limites de Segurança (Setpoints)

TEMP_INT_MIN, TEMP_INT_MAX = 15, 35

TEMP_EXT_MIN, TEMP_EXT_MAX = -50, 60

PRESSAO_MIN, PRESSAO_MAX = 120, 150

CARGA_MIN_PERCENT = 80



# === 2. ENTRADA DE DADOS (TELEMETRIA) ===

print("--- ENTRADA DE DADOS DE TELEMETRIA ---")

perc_atual = float(input("Porcentagem atual da carga (%): "))
```

```
temp_interna = float(input("Temperatura interna (°C): "))

temp_externa = float(input("Temperatura externa (°C): "))

pressao = float(input("Pressão do tanque (bar): "))

integridade_estrutural = int(input("Integridade estrutural (1-OK / 0-FALHA): "))


# Módulos Críticos

print("\nInforme o status dos módulos (OK ou FALHA):")

mod_nav = input("Módulo de Navegação: ").strip().upper()

mod_com = input("Módulo de Comunicação: ").strip().upper()

mod_prop = input("Módulo de Propulsão: ").strip().upper()


# === 3. PROCESSAMENTO E CÁLCULOS (ANÁLISE ENERGÉTICA) ===

# Cálculo baseado na relação Energia = Potência x Tempo e Eficiência

energia_disponivel = capacidade_total_kwh * (perc_atual / 100)

perda_energia = energia_disponivel * (taxa_perda_percent / 100)

energia_util = energia_disponivel - perda_energia

autonomia_restante = energia_util - consumo_decolagem_kwh


# === 4. ALGORITMO DE VERIFICAÇÃO (LÓGICA BOOLEANA) ===

# Verificação de faixas seguras

temp_safe = TEMP_INT_MIN <= temp_interna <= TEMP_INT_MAX

temp_ext_safe = TEMP_EXT_MIN <= temp_externa <= TEMP_EXT_MAX

pressao_safe = PRESSAO_MIN <= pressao <= PRESSAO_MAX

carga_safe = perc_atual >= CARGA_MIN_PERCENT

energia_suficiente = energia_util >= consumo_decolagem_kwh

integridade_ok = integridade_estrutural == 1
```

```
# Verificação dos módulos

nav_ok = mod_nav == "OK"

com_ok = mod_com == "OK"

prop_ok = mod_prop == "OK"

sistemas_criticos_ok = nav_ok and com_ok and prop_ok


# === 5. ANÁLISE ASSISTIDA POR IA (IDENTIFICAÇÃO DE ANOMALIAS) ===

alertas_risco = []

if 32 <= temp_interna <= 35:

    alertas_risco.append("ALERTA: Temperatura interna próxima ao limite
máximo (Risco de fadiga).")

if perc_atual < 85 and perc_atual >= 80:

    alertas_risco.append("CUIDADO: Margem de energia próxima ao limite mínimo
de segurança.")


# === 6. IMPRESSÃO DOS RESULTADOS ===

print("\n" + "="*30)

print("   RELATÓRIO FINAL DE VOO")

print("="*30)


# Exibição de Status Detalhado

print(f"Energia Útil: {energia_util:.2f} kWh")

print(f"Sobra após Decolagem: {autonomia_restante:.2f} kWh")

print(f"Status Estrutural: {'ÍNTEGRO' if integridade_ok else 'FALHA'}")


print("\n--- CHECKLIST DE SISTEMAS ---")

print(f"[ {'OK' if pressao_safe else '!!' } ] Pressão dos Tanques")

print(f"[ {'OK' if temp_safe else '!!' } ] Temperatura Interna")

print(f"[ {'OK' if carga_safe else '!!' } ] Nível de Carga")
```

```
print(f"[ {'OK' if sistemas_criticos_ok else '!!' } ] Módulos Críticos")


# Exibição de Alertas de Anomalias

if alertas_risco:

    print("\n--- ANÁLISE DE RISCO ---")

    for aviso in alertas_risco:

        print(aviso)


# VEREDITO FINAL

print("\n" + "="*30)

if (pressao_safe and temp_safe and temp_ext_safe and

    carga_safe and energia_suficiente and

    integridade_ok and sistemas_criticos_ok):

    print("STATUS: !!! PRONTO PARA DECOLAR !!!")

else:

    print("STATUS: !!! DECOLAGEM ABORTADA !!!")

    print("MOTIVO: Verifique as falhas apontadas no checklist acima.")

print("="*30)
```

3.2 Descrição das Funcionalidades

O script implementa três camadas de proteção:

1. **Validação de Tipo:** Garante que dados numéricos sejam processados corretamente para cálculos de autonomia.
2. **Normalização de Strings:** O uso de métodos como .upper() e .strip() previne falhas de interpretação caso o operador digite caracteres em formatos inesperados.
3. **Análise Preditiva:** O sistema não apenas verifica se o dado está "dentro ou fora" da faixa, mas emite alertas preventivos quando as variáveis se aproximam dos limites críticos, simulando uma assistência por inteligência artificial.

Resultado quando todos parâmetros estão dentro da normalidade:

```
--- ENTRADA DE DADOS DE TELEMETRIA ---
Porcentagem atual da carga (%): 81
Temperatura interna (°C): 30
Temperatura externa (°C): 32
Pressão do tanque (bar): 135
Integridade estrutural (1-OK / 0-FALHA): 1

Informe o status dos módulos (OK ou FALHA):
Módulo de Navegação: ok
Módulo de Comunicação: ok
Módulo de Propulsão: ok

==============================
   RELATÓRIO FINAL DE VOO
==============================
Energia Útil: 372.60 kWh
Sobra após Decolagem: 72.60 kWh
Status Estrutural: ÍNTEGRO

--- CHECKLIST DE SISTEMAS ---
[ OK ] Pressão dos Tanques
[ OK ] Temperatura Interna
[ OK ] Nível de Carga
[ OK ] Módulos Críticos

--- ANÁLISE DE RISCO ---
CUIDADO: Margem de energia próxima ao limite mínimo de segurança.

==============================
STATUS: !!! PRONTO PARA DECOLAR !!!
==============================
```

Resultado quando todos parâmetros estão dentro da normalidade, mas proximos dos limite definidos, onde a análise assistida por IA recomenda precaução:

```
--- ENTRADA DE DADOS DE TELEMETRIA ---
Porcentagem atual da carga (%): 81
Temperatura interna (°C): 30
Temperatura externa (°C): 32
Pressão do tanque (bar): 135
Integridade estrutural (1-OK / 0-FALHA): 1

Informe o status dos módulos (OK ou FALHA):
Módulo de Navegação: ok
Módulo de Comunicação: ok
Módulo de Propulsão: ok

==============================
   RELATÓRIO FINAL DE VOO
==============================
Energia Útil: 372.60 kWh
Sobra após Decolagem: 72.60 kWh
Status Estrutural: ÍNTEGRO

--- CHECKLIST DE SISTEMAS ---
[ OK ] Pressão dos Tanques
[ OK ] Temperatura Interna
[ OK ] Nível de Carga
[ OK ] Módulos Críticos

--- ANÁLISE DE RISCO ---
CUIDADO: Margem de energia próxima ao limite mínimo de segurança.

==============================
STATUS: !!! PRONTO PARA DECOLAR !!!
==============================
```

Resultado quando um dos parâmetros estão fora do esperado, onde a decolagem é cancelada:

```
--- ENTRADA DE DADOS DE TELEMETRIA ---
Porcentagem atual da carga (%): 79
Temperatura interna (°C): 25
Temperatura externa (°C): 35
Pressão do tanque (bar): 120
Integridade estrutural (1-OK / 0-FALHA): 1

Informe o status dos módulos (OK ou FALHA):
Módulo de Navegação: ok
Módulo de Comunicação: ok
Módulo de Propulsão: ok

==============================
   RELATÓRIO FINAL DE VOO
==============================
Energia Útil: 363.40 kWh
Sobra após Decolagem: 63.40 kWh
Status Estrutural: ÍNTEGRO

--- CHECKLIST DE SISTEMAS ---
[ OK ] Pressão dos Tanques
[ OK ] Temperatura Interna
[ !! ] Nível de Carga
[ OK ] Módulos Críticos

==============================
STATUS: !!! DECOLAGEM ABORTADA !!!
MOTIVO: Verifique as falhas apontadas no checklist acima.
==============================
```

4. ANÁLISE ENERGÉTICA E AUTONOMIA

A análise energética é um componente crítico para a viabilidade da Missão Aurora Siger. O sistema de gerenciamento de energia (EMS) da nave opera com base na relação fundamental entre potência e tempo, garantindo que haja reserva suficiente para a decolagem e para a manutenção dos sistemas de suporte à vida em órbita.

4.1 Fundamentação Matemática

Conforme os princípios da eficiência energética, a energia total consumida (E) é o produto da potência (P) pelo tempo de operação (t), expressa pela fórmula:

$$E = P.t$$

Onde as unidades de medida adotadas para este projeto são o **Quilowatt-hora (kWh)** para energia e o **Quilowatt (kW)** para demanda de potência.

4.2 Metodologia de Cálculo e Eficiência

Para o cálculo da autonomia real, o algoritmo implementado considera uma **Taxa de Perda de 8%**, proveniente de dissipação térmica e resistência nos condutores. O fluxo de cálculo segue as etapas abaixo:

1. **Energia Disponível (E_disp):** Calculada a partir da capacidade total (500 kWh) multiplicada pelo percentual de carga atual informado pela telemetria.
2. **Energia Útil (E_util):** Aplicação do fator de eficiência de 0,92 (considerando os 8% de perda).

$$E_util = E_disp * 0,92$$

3. **Margem de Segurança (M):** Subtração do consumo fixo de decolagem (300 kWh).

$$M = E_util - 300$$

4.3 Cenário de Simulação

Considerando uma leitura de telemetria com **85% de carga atual**, o sistema processa os seguintes valores:

- **Capacidade Total:** 500 kWh
- **Energia em Bateria:** 425 kWh (500 * 0,85$)
- **Energia Útil (pós-perdas):** 391 kWh (425 * 0,92)
- **Consumo na Decolagem:** 300 kWh
- **Sobra Operacional:** 91 kWh

Neste cenário, a missão é autorizada, pois a sobra operacional de **91 kWh** garante a estabilização da nave após o escape atmosférico. Caso a carga atual fosse inferior a **80%**, a energia útil resultante seria insuficiente para cobrir o consumo de decolagem e as perdas, acionando o cancelamento automático por critério energético.

5. ANÁLISE ASSISTIDA POR INTELIGÊNCIA ARTIFICIAL (IA)

A Missão Aurora Siger utiliza camadas de Inteligência Artificial para otimizar a tomada de decisão no computador de bordo. Diferente de sistemas convencionais que operam apenas com limites rígidos, a análise assistida permite identificar tendências e comportamentos que precedem uma falha catastrófica.

5.1 Classificação de Dados e Identificação de Anomalias

O sistema implementado no script Python utiliza uma lógica de **classificação** para monitorar a saúde dos componentes. Conforme os fundamentos de *Machine Learning* discutidos no Capítulo 6, o algoritmo atua na identificação de **Anomalias Operacionais** — dados que, embora ainda estejam dentro da faixa permitida, apresentam um comportamento inesperado ou perigoso.

No projeto, a IA monitora o "Gradiente Térmico". Se a temperatura interna ultrapassar **32°C**, o sistema dispara um alerta preventivo de "Risco de Fadiga", mesmo que o limite de cancelamento seja 35°C. Essa antecipação é característica de sistemas inteligentes de manutenção preditiva.

5.2 Sugestões de Risco e Tomada de Decisão

A integração da IA no checklist de decolagem oferece as seguintes vantagens:

1. **Redução de Falsos Positivos:** A IA pode correlacionar a temperatura externa com o consumo de energia para validar se o aquecimento é esperado ou se indica uma falha no sistema de refrigeração.
2. **Gestão de Incertezas:** Em cenários onde a integridade estrutural apresenta oscilações, modelos de aprendizado por reforço podem sugerir a melhor janela de lançamento para minimizar o estresse mecânico.
3. **Segurança Cognitiva:** Ao classificar o status dos módulos críticos como "OK" ou "FALHA" de forma autônoma, a IA reduz a carga cognitiva dos operadores humanos, permitindo que foquem em decisões estratégicas de alto nível.

A presença dessa camada de inteligência garante que a Missão Aurora não seja apenas automatizada, mas sim **autônoma**, capaz de aprender com o histórico de telemetria para aumentar as taxas de sucesso em lançamentos futuros.

6. REFLEXÃO CRÍTICA: ÉTICA, SOCIEDADE E SUSTENTABILIDADE

A exploração espacial e a gestão de infraestruturas de missão crítica representam um dos maiores desafios da engenharia contemporânea, exigindo uma integração profunda entre o desenvolvimento técnico e a responsabilidade ética. A execução desta atividade, centrada no sistema de telemetria da Missão Aurora Siger, permite analisar como a tecnologia deve servir ao propósito do desenvolvimento sustentável e à segurança social.

6.1 Ética e Responsabilidade Social (ISO 26000)

De acordo com as diretrizes da **NBR ISO 26000**, a responsabilidade social organizacional manifesta-se na transparência e na prestação de contas pelos impactos de suas decisões. No sistema desenvolvido, o "Fator Humano" — representado pela validação obrigatória dos módulos de navegação e integridade estrutural no script Python — assegura que a decisão final de decolagem não seja delegada exclusivamente a algoritmos de "caixa-preta".

Esta abordagem mitiga riscos de falhas automatizadas silenciosas e garante que o julgamento ético prevaleça sobre a eficiência técnica pura. Ao implementar alertas de anomalia (como o risco de fadiga térmica acima de 32°C), o sistema prioriza a salvaguarda da vida humana e do patrimônio tecnológico em detrimento da pressa operacional.

6.2 Sustentabilidade e o Triple Bottom Line (TBL)

A análise da missão sob a ótica do **Triple Bottom Line (TBL)** revela a interdependência entre os pilares ambiental, social e econômico:

- **Pilar Ambiental:** A precisão matemática no cálculo de autonomia e das perdas energéticas de 8% reflete a aplicação de conceitos de **TI Verde (Green IT)**. O objetivo é minimizar o desperdício de recursos e a pegada de carbono da operação através da otimização algorítmica.
- **Pilar Social:** As tecnologias de monitoramento desenvolvidas para missões críticas geram inovações em sistemas embarcados que, historicamente, transbordam para a segurança da sociedade civil (como em aviação comercial e medicina).
- **Pilar Econômico:** A eficiência energética detalhada na Fase 4 garante a viabilidade financeira de projetos de longo prazo, transformando a sustentabilidade em um diferencial estratégico e reduzindo custos operacionais desnecessários.

6.3 Sustentabilidade Tecnológica e Green IT

Conforme os preceitos de sustentabilidade tecnológica, a **TI Verde** foca na redução do impacto ambiental ao longo de todo o ciclo de vida dos ativos. No contexto de software, isso se traduz na implementação de códigos otimizados que demandam menor capacidade de processamento e, consequentemente, menor consumo energético dos servidores de monitoramento.

A sustentabilidade, portanto, não se limita ao propelente da aeronave, mas estende-se à infraestrutura de hardware e software. Ao evitar a obsolescência acelerada e priorizar a eficiência global do sistema de telemetria, o projeto alinha-se às metas globais de desenvolvimento sustentável.

7. REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO 26000**: Diretrizes sobre responsabilidade social. Rio de Janeiro: ABNT, 2010.

ELKINGTON, John. **Cannibals with Forks**: The Triple Bottom Line of 21st Century Business. Oxford: Capstone, 1997.

FIAP. **Capítulo 1: Mercado e Tecnologia - As Forças Sociais e Tecnológicas que Moldam o Mundo Digital**. São Paulo: FIAP, 2026.

FIAP. **Capítulo 4: Os Primeiros Traços que Estruturam o Pensamento Algorítmico**. São Paulo: FIAP, 2026.

FIAP. **Capítulo 5: As Estruturas Lógicas que Sustentam as Primeiras Escolhas da Missão**. São Paulo: FIAP, 2026.

FIAP. **Capítulo 6: O Surgimento da Inteligência que Apoia a Tomada de Decisão**. São Paulo: FIAP, 2026.

FIAP. **Capítulo 7: A Energia que Sustenta o Primeiro Impulso da Missão**. São Paulo: FIAP, 2026.

FIAP. **LIVE 17/03/2026: Telemetria Energética - Conceitos, Coleta de Dados e Exemplos em Python**. São Paulo: FIAP, 2026.

PYTHON SOFTWARE FOUNDATION. **Python Language Reference, version 3.12**. Disponível em: https://docs.python.org/. Acesso em: 20 mar. 2026.